E7: Introduction to Computer Programming
Final Exam – Version A

Thursday, December 14

Full Name

Student ID Number

Instructions:

- This includes both Part I (Multiple Choice) AND Part II (Fill in the Blanks).
- Write your name above AND on the bubble sheet.
- Mark the correct version letter on the bubble sheet.
- This exam is DOUBLE-SIDED. Make sure to read both sides of each page.
- You have 3 hours to complete this exam.
- You may use two double-sided sheets of notes.
- The exam is worth a total of 300 points.
- For questions with circular bubbles, you should select exactly *one* choice.
  - ◯ You may only choose one option!
  - ◯ Or this one, but not both!
  - ◯ Or this one, but no more than one!
- For questions with square boxes, you may select *multiple* choices.
  - ☐ You can select this one!
  - ☐ You can select this one too!
  - ☐ You can select one, two, or all three of these!
- Only include your FINAL answers in the designated boxes. Any additional text will count as part of your answer and may result in a reduced grade.
- At the end of the exam, hand in the completed bubble sheet AND the exam.

Good luck!

Question 1 (150 points)

There are 50 multiple choice questions, each worth 3 points, for a possible total of 150 points. There is **ONE and only one** correct choice for each question.

1. The central difference method is known as 'second order' because

   A. it takes more time to run
   B. it results in high error
   C. if we divide the step size by 2, the error decreases by a factor of 2
   D. the derivative is computed by dividing the difference by twice the step size
   E. if we divide the step size by 2, the error decreases by a factor of 4

2. After running the following Python code, what will be the final value of `pos`?

```
import numpy as np
pos = np.arange(-4, 4)
pos[pos >= 2] = pos[pos >= 2] - 2
```

   A. `[-4 -3 -2 -1 0 1 2 1 2]`
   B. None of the options
   C. `[-4 -3 -2 -1 0 1 0 1 2]`
   D. `[-4 -3 -2 -1 0 1 2 1]`
   E. `[-4 -3 -2 -1 0 1 0 1]`

3. Assume that the MATLAB code below has been executed.

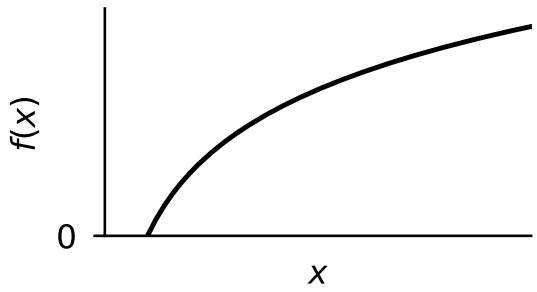
```
A = [1, 0; 2, 3; 5, 4];
B = [1, 0, 0; 0, 1, 0];
```

   Which of the following MATLAB expressions will NOT result in an error?

   A. `A .* B`
   B. `A - B`
   C. None of the options result in an error
   D. `A + B`
   E. `A * B`

4. For the function $f(x)$ shown below, which differentiation method will likely yield the highest (i.e., largest) numerical estimate of the derivative of the function at any point?

   A. Forward difference
   B. Backward difference
   C. Central difference
   D. Midpoint rule
   E. Simpson's rule



5. You need to perform a numerical computation, which uses a step size h. You've come across various methods that exhibit distinct orders of error. Which method would you select?

   A. Method with $O(h)$
   B. Method with $O(h^4)$
   C. Method with $O(h^3)$
   D. Method with $O(h^2)$
   E. Method with $O(\log h)$

6. After running the following Python code, what will be the final value of `var`?

```
var = 2
var += 2 ** 4 / 2
```

   A. 4
   B. 10
   C. 8
   D. 2
   E. 6

7. Let `y` and `t` be two NumPy arrays of the same size. Which numerical differentiation method does the following function correspond to?

```
def numerical_differentiation(y, t):
    return (y[1:] - y[:-1]) / (t[1:] - t[:-1]), t[1:]
```

   A. Central difference
   B. Forward difference
   C. Simpson's rule
   D. Backward difference
   E. Midpoint rule

8. While using ChatGPT to better understand root finding, it mentioned the following sentence, *"Rather than aiming for precisely zero error, numerical algorithms employ tolerance thresholds to decide when to terminate the iterations."* This statement is

   A. true because numerical root finding methods were developed more than 200 years ago
   B. false
   C. true because the algorithm must account for truncation errors
   D. true because the algorithm must account for round-off errors
   E. true because ChatGPT is always correct

9. Which of the following is True regarding numerical differential equation solvers?

   A. They are not suitable for systems of differential equations
   B. They cannot be used for solving higher order differential equations
   C. Increasing the step size increases accuracy
   D. They are iterative in nature
   E. Increasing the step size increases computation time

10. Round-off errors occur because numbers in computers are

    A. represented using a fixed number of bits
    B. represented using the binary system
    C. represented using the decimal system
    D. truncated from an infinite to a finite sum
    E. A and B

11. What will the expression `int(5.8) + float(4.5)` evaluate to in Python?

    A. 10.3
    B. 9.5
    C. 10.5
    D. 10
    E. 9

12. The decay of a radioactive isotope, can be modeled by the following differential equation:

$$\frac{d^2N}{dt^2} + 100\frac{dN}{dt} + N = 0$$

    where N is the concentration of the isotope at a given time. Let the state be: $S = \begin{bmatrix} N \\ N' \end{bmatrix}$

    Which of the following is Forward Euler's solution with a step size of 0.1 in matrix form?

    A. $\begin{bmatrix} N_{i+1} \\ N'_{i+1} \end{bmatrix} = \begin{bmatrix} 0 & 0.1 \\ -0.1 & -10 \end{bmatrix} \begin{bmatrix} N_i \\ N'_i \end{bmatrix}$

    B. $\begin{bmatrix} N_{i+1} \\ N'_{i+1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ -1 & -99 \end{bmatrix} \begin{bmatrix} N_i \\ N'_i \end{bmatrix}$

    C. $\begin{bmatrix} N_{i+1} \\ N'_{i+1} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1 & -100 \end{bmatrix} \begin{bmatrix} N_i \\ N'_i \end{bmatrix}$

    D. $\begin{bmatrix} N_{i+1} \\ N'_{i+1} \end{bmatrix} = \begin{bmatrix} 1 & 0.1 \\ -0.1 & -9 \end{bmatrix} \begin{bmatrix} N_i \\ N'_i \end{bmatrix}$

    E. $\begin{bmatrix} N_{i+1} \\ N'_{i+1} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} N_i \\ N'_i \end{bmatrix}$

13. You want to compute $I = \int_0^5 f(x) = xe^{x^2}$ using `scipy.integrate.quad(func, a, b, ...)`, where `func` is a function object of the function you want to integrate and `a, b` are two scalar values representing the lower and upper limits of integration, respectively. The function also takes additional optional arguments. What is wrong, if anything, with the code below?

```
import scipy
import numpy as np
def func(x):
    print(x * np.exp(x ** 2))
I = scipy.integrate.quad(func, 0, 5)
```

A. `func` should have `return` instead of `print()`

B. Nothing is wrong with the code (it will correctly compute I)

C. `scipy.integrate.quad` should have an additional argument `args=(...)`

D. The code has incorrect indentation

E. The mathematical expression `x * np.exp(x ** 2)` has wrong order of precedence

14. The Bisection method is a numerical method that is used to

A. integrate a function

B. differentiate a function

C. find the maximum value of a function

D. optimize functions

E. find a root of a function

15. Programming makes your work

A. transparent

B. reproducible

C. All of the options

D. shareable

E. efficient

16. Which of the following is False regarding the Newton-Raphson method?

A. It requires knowing or estimating the derivative of the function

B. It will converge to the root closest to the initial guess

C. It is used to estimate roots of a function

D. It may converge to a different root than the desired one

E. In general, it converges to a solution faster than the Bisection method

17. Regression is used to find

    A. data points that are as close as possible to the actual data

    B. a curve that best fits the data by minimizing some measure of error

    C. a curve that best fits the data by maximizing some measure of error

    D. the values at which a function is equal to zero

    E. a curve that exactly fits the data

18. Consider the following Python code, where `a` is a data structure.

```
s = 0
for i in range(len(a)):
    if a[i] > 2:
        s += 1
```

    Which of the following options is equivalent to the above code? Note that `sum` is a built-in function that returns the sum of the elements in a data structure.

    A.
```
s = sum(a > 2)
```

    B.
```
s = sum([2 for i in a if i > 1])
```

    C.
```
s = lambda a: sum(a > 2)
```

    D.
```
s = 0
for i in a:
    if a[i] > 2:
        s += 1
```

    E.
```
s = 0
for i in range(len(a)):
    if a[i] > 2:
        s + 1
```

19. Let `f` be a function object you want to integrate and `a, b, h` be three scalar values representing the lower integration bound, upper integration bound, and step size, respectively. Which numerical integration method does the following function correspond to?

```
def numerical_integration(f, a, b, h):
    x = np.arange(a, b, h)
    return np.sum(h * f(x + h/2))
```

    A. Left or Right Riemann (either one)

    B. Midpoint rule

    C. None of the methods

    D. Trapezoid rule

    E. Simpson's rule

20. Which of the following analogies was given in the lecture when introducing functions?

    A. Functions are like televisions

    B. Functions are like cars

    C. Functions are like pizza

    D. Function are like toasters

    E. Functions are like refrigerators

21. Your friend wrote the root finding function below, which takes as input a function object `f`, two scalar values `a, b` representing an initial range, and a scalar value `tol` representing the tolerance. The absolute value of the function at the acceptable root should be less than `tol`. What is wrong, if anything, with the function? Note that line numbers are shown to the left.

```
1    import numpy as np
2    def roots(f, a, b, tol):
3        assert f(a) * f(b) < 0, 'Invalid [a, b]'
4        midpoint = (a + b) / 2
5        while np.abs(f(midpoint)) >= tol:
6            if f(midpoint) * f(a) >= 0:
7                a = midpoint
8            else:
9                b = midpoint
10       return midpoint
```

    A. Nothing is wrong with the function

    B. The logical expression on line 5 is incorrect

    C. The assert statement on line 3 is incorrect

    D. There is a missing line of code that should be added after line 9

    E. The assignment expressions on lines 7 and 9 should be swapped

22. Consider the following MATLAB code:

```
midterm_1 = [95.5, 49, 89.5, 71.5, 43];
midterm_2 = [86.5, 78, 90.5, 71.5, 78];
compare = (midterm_1 < midterm_2) & (midterm_1 < 50);
```

    Which option yields an output equivalent to that of `compare` in the code above?

    A. `compare = (midterm_1 < midterm_2) .* (midterm_1 < 50);`

    B. `compare = (midterm_1 < midterm_2) + (midterm_1 < 50);`

    C. `compare = (midterm_1 < midterm_2) and (midterm_1 < 50);`

    D. `compare = (midterm_1 < midterm_2) * (midterm_1 < 50);`

    E. `compare = (midterm_1 < midterm_2) && (midterm_1 < 50);`

23. Your friend, who hasn't taken E7, is trying to solve a system of 5 linear equations with 4 unknowns: x_1, x_2, x_3, x_4. After overhearing you while watching the E7 recordings, they believe they know how to express this system in matrix form and came up with the form shown below. How can you fix the matrix form of this system of linear equations?

$$\underbrace{\begin{bmatrix} 1 & -2 & 0 & 0 \\ 1 & 0 & 0 & 3 \\ 0 & 0 & 1 & -1 \\ 0 & 1 & 1 & 0 \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} 100 \\ 10 \\ 90 \\ 70 \\ 50 \end{bmatrix}}_{\mathbf{b}}$$

A. Add one more column to matrix **A**

B. Add one more row to matrix **A**

C. Remove one row from vector **b**

D. None of the options (i.e., the system is correctly defined)

E. Add one more row to vector **x**

24. The accuracy of a truncated Taylor series approximation can be improved by

A. using more subintervals

B. using a smaller step size

C. using more terms

D. A and B

E. A, B, and C

25. What will be displayed after running the following Python code?

```
def fun(a, b):
    if a < b:
        return a - b
    else:
        return fun(a-2, b)

print(fun(5, 2))
```

A. Error

B. 1

C. -1

D. 3

E. None of the options

26. You defined a system of linear equations in MATLAB and obtained the following results.

```
>> A = [2, 1, 2; 1, -2, -1; 4, 0, -1];
>> b = [6; 5; 8];
>> rank_A = rank(A)
rank_A =
       3

>> rank_Ab = rank([A, b])
rank_Ab =
       3
```

If the matrices are correctly defined, this system of linear equations has

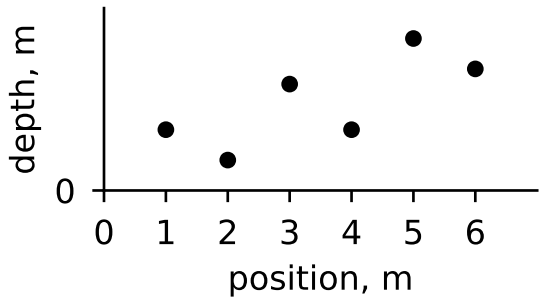
A. It is not possible to determine anything about the solution based on the given information
B. no solution
C. a solution but it is not possible to determine whether it is unique or not based on the given information
D. a unique solution
E. infinite solutions

27. You collected a data set of temperature measurements taken every 15 minutes. You are interested in analyzing this data, which might require unit conversion and other numerical computations. What is the recommended data structure for storing this data set in Python?

A. Range
B. List
C. NumPy Array
D. Tuple
E. String

28. You've collected a data set containing the floor depth of a lake at various positions along a specific cross-section, as shown in the figure below. To numerically integrate this data with a step size of 1 m, which, if any, of the following methods cannot be used?

A. None of the options (i.e., all can be used)
B. Simpson's rule
C. Midpoint rule and Simpson's rule
D. Midpoint rule
E. Left and Right Riemann

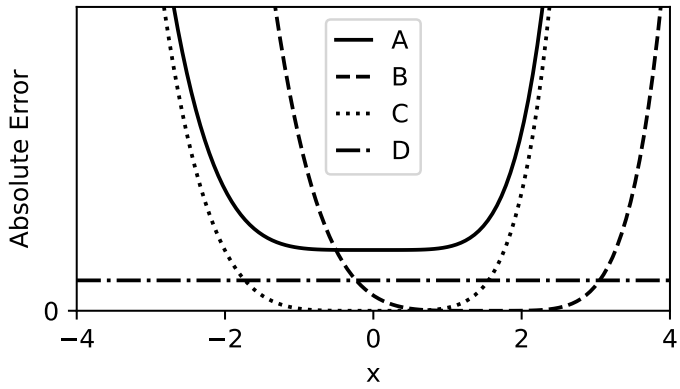


29. What is the highest number that can be represented with a 3-bit signed binary representation?

    A. 4
    B. 3
    C. 2
    D. 7
    E. 14

30. After computing the Taylor series expansion for $f(x) = e^x$ around the point $a = 0$, you plotted the absolute value of the error as a function of x. Which of the curves in the figure below represents the correct trend of the absolute error?

    A. A
    B. B
    C. C
    D. D
    E. None of the options



31. How can the error of numerical integration methods be reduced, and what is the resulting consequence?

    A. Decrease step size, increase computation time
    B. It depends on the integration method
    C. Increase step size, decrease computation time
    D. Decrease step size, decrease computation time
    E. Increase step size, increase computation time

32. The Van der Pol equation describes the motion of dynamic systems as:

$$\frac{d^2x}{dt^2} - \mu\left(1 - x^2\right)\frac{dx}{dt} + x = 0$$

    where μ is some constant. What can you say about the Van der Pol equation?

    A. None of the options
    B. It is a second order ordinary differential equation
    C. It is a third order ordinary differential equation
    D. It is a third order partial differential equation
    E. It is a second order partial differential equation

33. What will be displayed after running the following Python code?

```
import numpy as np
midterm_1 = np.array([95.5, 49, 89.5, 71.5, 43])
midterm_2 = np.array([86.5, 78, 90.5, 71.5, 78])
print(midterm_1 < midterm_2)
```

A. `[False True True True True]`

B. `[False True True False True]`

C. `False`

D. None of the options

E. `[True False False True False]`

34. For the function $f(x)$ shown below, which integration method will likely yield the highest (i.e., largest) numerical estimate of the area under the curve?

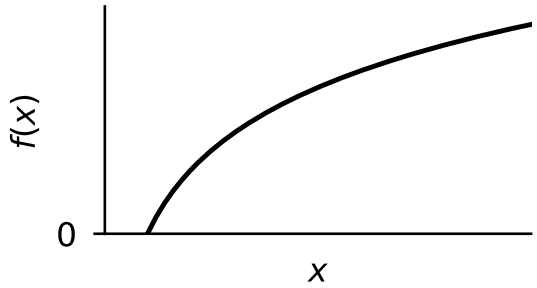
A. Left Riemann

B. Midpoint rule

C. Simpson's rule

D. Right Riemann

E. Trapezoid rule



35. Assume that the following MATLAB function, `fun()`, has been defined.

```
function [c] = fun(a, b)
c = 0;
if a > b
c = a - b;
end
if c < 10
return
else
c = 10;
end
end
```

What will be the output of `fun(5, 2)`?

A. 0

B. 10

C. 3

D. None of the options

E. -3

36. Assume `wind` is a list that includes wind speeds. Consider the following Python code.

```
m = len(wind)
for i in range(m):
    wind[i] *= 0.3048
```

Which of the following options is equivalent to the above code?

A.
```
for value in wind:
    value *= 0.3048
```

B.
```
m = len(wind)
for i in len(m):
    wind[i] *= 0.3048
```

C.
```
m = len(wind)
for i in m:
    wind[i] *= 0.3048
```

D.
```
for j, value in enumerate(wind):
    wind[j] = value * 0.3048
```

E.
```
m = len(wind)
for i in range(m):
    wind[i] = 0.3048
```

37. What will be displayed after running the following Python code?

```
def physics(mass, net_force):
    if mass > 1:
        force = 10 * mass
        friction = net_force - force
        if friction > 0:
            return friction
        else:
            return 'Negative friction detected'
    else:
        return 0

print(physics(2, 10))
```

A. Error

B. `'Negative friction detected'`

C. `0`

D. `-10`

E. `10`

38. Which of the following is NOT true regarding differences between Python and MATLAB?

    A. `[9, 1, 5]` is a `list` in Python but a `double array` in MATLAB

    B. Python uses 0-based indexing whereas MATLAB uses 1-based indexing

    C. Many mathematical functions are built into MATLAB but need to be imported in Python

    D. Indentation is required in Python but optional in MATLAB

    E. None of the options (i.e., all are True)

39. What will be displayed after running the following Python code?

```
def fun(a, b):
    if a > b:
        c = a - b
    else:
        c = b - a
    return c
fun(5, 2)
print(c)
```

    A. 3

    B. -2

    C. 5

    D. 2

    E. Error

40. After running the following Python code, what will be the final value of `f`?

```
m = 5
a = 2
if a == 2:
    f = m * a
    a = 10
if a == 10:
    m = 10
    f = f / 10
elif m == 10:
    f = 2
```

    A. `f` will not be defined

    B. 1

    C. 5

    D. 10

    E. 2

41. What happens if a recursive function does not have any base case(s)?

    A. The function will have an infinite loop
    B. The function will be iterative
    C. The function will have an infinite recursion
    D. The function will return an incorrect answer
    E. The function will return a correct answer but it will take too long

42. What was the secondary programming language used in E7 this semester?

    A. JavaScript
    B. LOLCODE
    C. Python
    D. MATLAB
    E. C++

43. What was the primary programming language used in E7 this semester?

    A. JavaScript
    B. C++
    C. LOLCODE
    D. Python
    E. MATLAB

44. Which of the following errors is certain to lead a program to crash?

    A. Syntax error
    B. Round-off error
    C. Truncation error
    D. Logic error
    E. Runtime error

45. Which of the following is not a valid number in base 4 representation?

    A. 00000
    B. 10401
    C. 10001
    D. 32221
    E. 12003

46. What will be displayed after running the following Python code?

```
data1 = [0, 1, 1, 0, 1]
data2 = [2, 2]
print(data1 + data2)
```

A. `[0, 1, 1, 2, 3]`
B. Error
C. `[2, 3, 1, 0, 1]`
D. `[0, 1, 1, 2, 2]`
E. `[0, 1, 1, 0, 1, 2, 2]`

47. Order the following integration methods from the most accurate (least error) to least accurate (highest error) for the same step size h:

A. Simpson's, Trapezoid, Left Riemann
B. Trapezoid, Simpson's, Left Riemann
C. Simpson's, Left Riemann, Trapezoid
D. Left Riemann, Trapezoid, Simpson's
E. Trapezoid, Left Riemann, Simpson's

48. Which error among the following options is not caused by incorrectly written code?

A. None of the options (i.e., all are caused by incorrectly written code)
B. Logic error
C. Round-off error
D. Runtime error
E. Syntax error

49. Which of the following can be effectively solved using numerical methods?

A. Finding roots to algebraic equations
B. Finding the integral of a function
C. Finding solutions to differential equations
D. All of the options
E. Finding the derivative of a function

50. What is the preferred method for implementing code that performs the same task on an input?

A. Using iteration and conditional statements
B. Using recursion
C. Using iteration
D. Using functions
E. Using conditional statements

Question 2: Kitchen Alcatastrophe (30 points)

James A. isn't the greatest cook. Based on data collected over one week, we are interested in estimating the probability that a meal he cooks is edible, which we will refer to as an unknown x. Based on the collected data, we developed the following relationship: $e^{-x} + xe^{-x} = 0.8$

One challenge is that the exponential function cannot be directly evaluated. The exponential function can be approximated using Taylor series around $a = 0$ as:

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$$

Complete the following Python function which takes as input a scalar `x` and a positive integer `N` and returns `est`, which is an approximation of e^x based on the above equation up to $n = N$. Assume you have defined a function called `factorial(n)`, which returns the factorial of an integer n.

```
def exp(x, N):
    """Return an approximate value of the exponential of x up to n=N.
    >>> exp(1, 1)
    2.0
    >>> exp(1, 2)
    2.5
    """
    ______(a)______
    for n in ______(b)______:
        ______(c)______
    return est
```

(a) (3 pt) Fill in blank (a).

(b) (3 pt) Fill in blank (b).

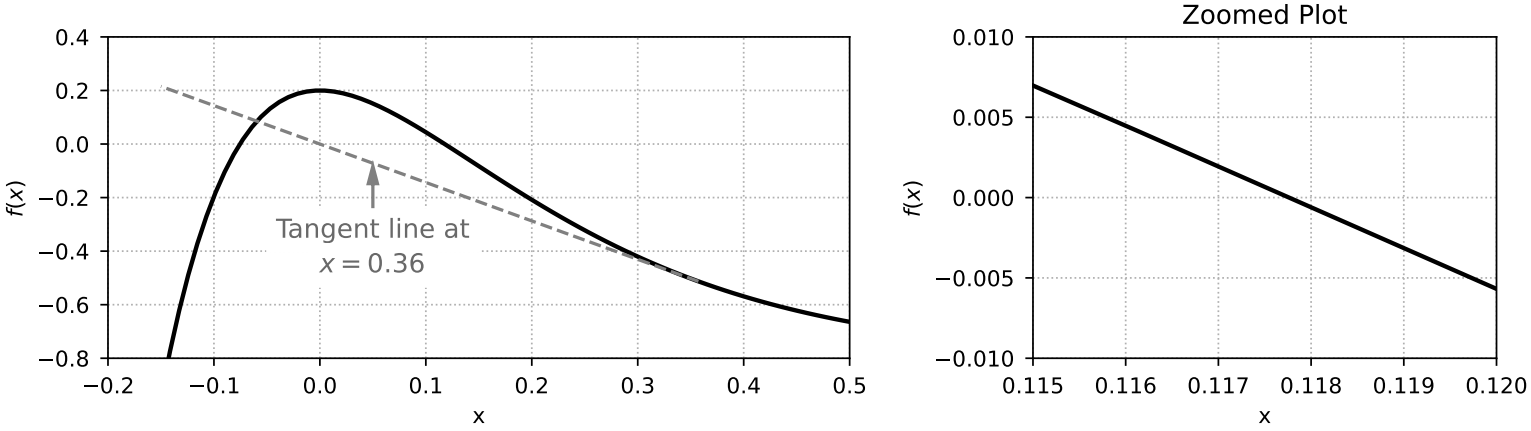
```
for n in
```

(c) (4 pt) Fill in blank (c).

We are interested in solving the equation $e^{-x} + xe^{-x} = 0.8$ to determine the probability that a cooked meal is edible. The next step is to rewrite this equation in the form $f(x) = 0$, where $f(x)$ is the function we are trying to find its root.

(d) (6 pt) Write a lambda function `f(x, N)` that takes as input a scalar `x` and a positive integer `N` and returns the value of the function $f(x)$ evaluated at the input `x`. Use your function `exp()` to approximate any exponential value up to $n = N$.

After defining the above function, you used it to plot $f(x)$ and a zoomed plot as shown below.



(e) (5 pt) Assume that an acceptable estimate of a root is when the value of the function at that estimate is within a tolerance equal to 0.005. Which of the following values is(are) considered acceptable estimates of the probability that a cooked meal is edible? Select ALL that apply.

☐ 0.115	☐ 0.117	☐ 0.119	☐ 0.000	☐ −0.005
☐ 0.116	☐ 0.118	☐ 0.120	☐ 0.005	☐ −0.100

(f) (5 pt) Using the Newton-Raphson method, which of the following initial guesses can be used to estimate the root between 0.10 and 0.20? Select ALL that apply.

☐ −0.1	☐ 0.0	☐ 0.10	☐ 0.36
☐ −0.05	☐ 0.05	☐ 0.20	☐ 0.40

(g) (4 pt) Which of the following is(are) True about the Bisection method? Select ALL that apply.

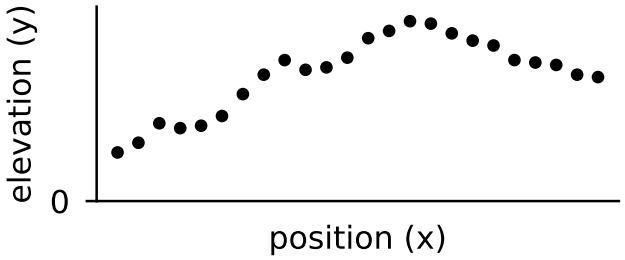
- ☐ Increasing the tolerance will generally increase the accuracy of the estimated root
- ☐ Increasing the tolerance will generally increase the time needed to find an acceptable root
- ☐ For the above example, an initial range of $[0, 0.36]$ will yield an acceptable root
- ☐ It will always converge to a root that is between an acceptable initial range
- ☐ Choosing a narrow initial range improves the accuracy of the estimated root
- ☐ For the above example, an initial range of $[-0.1, 0.4]$ will yield an acceptable root

Question 3: Hills, Thrills, and Tunnel Spills (30 points)

You work for a company that does heavy civil construction. Your upcoming project is to design a tunnel aimed at reducing traffic. The challenge is that the tunnel has to cross through the Berkeley Hills. The first step is to look at a cross-section of the Berkeley Hills and to perform calculations to better understand how steep the hills are, how much soil and rock is beneath them, etc.

You collected a data set of the ground elevation at different positions across the hills, as shown to the right. You then imported the data into Python as two NumPy arrays:

- `x`: array of horizontal positions
- `y`: array of corresponding ground elevations



First, we are interested in estimating the steepness of the hills at different positions. Steepness is simply the slope of the ground, which can be estimated using the derivative.

(a) (3 pt) Which of the following methods would you select to estimate the steepness of the hills?

◯ numerical integration	◯ analytic differentiation	◯ Taylor series
◯ numerical differentiation	◯ analytic root finding	◯ interpolation
◯ analytic integration	◯ numerical root finding	◯ regression

Complete the Python function below which takes as input two NumPy arrays `y, x` of the elevation data and their corresponding positions, respectively, and returns two NumPy arrays `slope, pos` representing the derivative values (i.e., slope of the ground) and their corresponding positions using the forward difference method. You should implement this function without using iteration. Note that `x` may have variable spacing.

```
def steepness(y, x):
    slope = ______(b)______
    pos = ______(c)______
    return slope, pos
```

(b) (5 pt) Fill in blank (b).

```
slope =
```

(c) (3 pt) Fill in blank (c).

```
pos =
```

(d) (5 pt) Which of the following options could improve the accuracy of the estimated steepness? Select ALL that apply.

- ☐ Using central difference on the same data
- ☐ Using backward difference on the same data
- ☐ Using trapezoid rule on the same data
- ☐ Using forward difference on the same data but with a smaller step size
- ☐ Using forward difference on new data obtained with a smaller step size

(e) (3 pt) Next, you want to estimate how much soil and rock is beneath the hills, which is simply the area between the ground elevation and sea level elevation (elevation of 0). Which of the following methods would you select?

◯ numerical integration	◯ analytic differentiation	◯ Taylor series
◯ numerical differentiation	◯ analytic root finding	◯ interpolation
◯ analytic integration	◯ numerical root finding	◯ regression

Complete the Python function below which takes as input two NumPy arrays `y, x` of the elevation data and their corresponding positions, respectively, and returns a scalar value `area` representing the area computed using the trapezoid rule from the first to the last value in array `x`. You should implement this function using iteration. Note that `x` may have variable spacing.

```
def soil_rock(y, x):
    area = ______(f)______
    for i in ______(g)______:
        ______(h)______
    return area
```

(f) (3 pt) Fill in blank (f).

```
area =
```

(g) (3 pt) Fill in blank (g).

```
for i in
```

(h) (5 pt) Fill in blank (h).

```

```

Question 4: ROON-guh KOOT-uh Tripple Trouble (30 points)

A first order differential equation has the general form $y' = F(t, y)$, where F is some function of t and y. For example, the temperature y of a cooling hot object at a given time t can be modeled as:

$$y' = -0.005\,(y - 25)^2$$

(a) (5 pt) Write a Python function `cool(t, y)` that takes as input two scalar values representing the current time and the current temperature, respectively, and returns the value of y' based on the above equation. Both the function definition and body should be a *single line of code*.

While not discussed in class, you will implement the third-order Runge Kutta method to solve this differential equation. The method is based on evaluating three slopes k_1, k_2, k_3. Then, using a current point (t_i, y_i) and a weighted average of the three slopes, the value at the next time step t_{i+1} can be computed. The equations are shown below, where h is the step size.

$$\begin{cases} k_1 = F(t_i,\ y_i) \\ k_2 = F\left(t_{i+1/2},\ y_i + \dfrac{h}{2}k_1\right) \\ k_3 = F\left(t_{i+1},\ y_i - hk_1 + 2hk_2\right) \end{cases} \qquad y_{i+1} = y_i + \frac{h}{6}\left(k_1 + 4k_2 + k_3\right)$$

Write a Python function `RK3`, which takes as input a function object `F` of the right hand side of the differential equation $F(t, y)$, and four scalar values `t0, y0, tf, h` representing the initial time, the initial condition, the final time at which the solution is desired, and the step size, respectively. The function should only return the estimated solution of y at the final time `tf` using the third-order Runge Kutta method. Assume that the input `tf` is some multiple of `h` plus the initial time `t0`.

```
def RK3(F, t0, y0, tf, h):
    ti = t0       # current time step
    yi = y0       # solution at current time step
    while ______(a)______:
        k1 = ______(b)______
        k2 = ______(c)______
        k3 = ______(d)______
        yi = ______(e)______
        ti += h
    return yi
```

You will implement the function `RK3` by filling in the blanks. You may only write one statement on each line. You may not use ; to place two statements on the same line. You may not use ANY functions or methods, built-in or otherwise, except for the input function `F`. You may not modify the indentation of any line.

(a) (3 pt) Fill in blank (a).

```
while
```

(b) (3 pt) Which of the following options could fill in blank (b)?

- ◯ `F(ti, yi)`
- ◯ `F(t[i], y[i])`
- ◯ `F(ti[i], yi[i])`
- ◯ `F(ti, yi[0])`
- ◯ `F(t0, y0)`
- ◯ `F(t, y)`
- ◯ `F(ti, yi[-1])`
- ◯ `F(ti[1], yi[-1])`
- ◯ `F(ti[0], yi[0])`

(c) (3 pt) Fill in blank (c).

```
k2 =
```

(d) (3 pt) Fill in blank (d).

```
k3 =
```

(e) (5 pt) Fill in blank (e).

```
yi =
```

(f) (5 pt) The temperature of a cooling hot object is modeled using the differential equation shown on the previous page. The object has a temperature of $100°C$ at $t = 0$. Write *one line of code* to estimate its temperature at $t = 1$ hr using your function `RK3` with time steps of 0.01 hr.

(g) (3 pt) We would like to set the initial time `t0` in function `RK3` to have a default value of 0. Complete the line below so that the initial time `t0` has a default value of 0.

```
def RK3
```

Question 5: Fibonacci but on Steroids (20 points)

A "k-bonacci sequence" is a generalization of the Fibonacci sequence. In a k-bonacci sequence, each term is the sum of the k preceding terms. The Fibonacci sequence is a special case with k = 2, where each term is the sum of the two previous terms. In a 3-bonacci sequence (k = 3), each term is the sum of the three previous terms, and so on. A k-bonacci sequence starts with k − 1 zeros and then a one. Each subsequent element is the sum of the previous k elements. For example:

```
       n: 0, 1, 2, 3, 4, 5, 6,  7,  8,  9, ..., n
3-bonacci: 0, 0, 1, 1, 2, 4, 7, 13, 24, 44, ..., F(n-1) + F(n-2) + F(n-3)
4-bonacci: 0, 0, 0, 1, 1, 2, 4,  8, 15, 29, ..., F(n-1) + F(n-2) + F(n-3) + F(n-4)
```

Hint: Using this information, try to come up with the k-bonacci formula for different values of n.

Write a function `kbonacci()` which takes a non-negative integer `n` and a positive integer `k`. The function should return the value of the nth element of the k-bonacci sequence. Anything written on this page will NOT be graded. Only answers within the designated boxes on the next page will be graded. The function below is written in Python and your answers should use Python syntax.

```
def kbonacci(n, k):
    """Return element n of a k-bonacci sequence.

    >>> kbonacci(8, 3)
    24

    >>> kbonacci(9, 3)
    44

    >>> kbonacci(9, 4)
    29
    """

    if n < k - 1:

        return ______(a)______

    elif ______(b)______:

        return 1

    else:

        total = 0

        i = ______(c)______

        while i < n:
            total += ______(d)______

            i += 1

# return statement has been omitted. You will add it in part (e).
```

You will implement the function `kbonacci()` by filling in the blanks. You may only write one statement on each line. You may not use ; to place two statements on the same line. You may not use any built-in functions or methods. You may not modify the level of indentation of any line.

(a) (4 pt) Fill in blank (a). Use Python syntax for all parts on this question.

```
return
```

(b) (4 pt) Fill in blank (b).

```
elif
```

(c) (4 pt) Fill in blank (c).

```
i =
```

(d) (4 pt) Fill in blank (d).

```
total +=
```

(e) (4 pt) Which of the following can be used to return the correct output?

◯
```
def kbonacci(n, k):
    if n < k - 1:
        return ______(a)______
    elif ______(b)______:
        return 1
    else:
        total = 0
        i = ____(c)____
        while i < n:
            total += ____(d)____
            i += 1
        return total
```

◯
```
def kbonacci(n, k):
    if n < k - 1:
        return ______(a)______
    elif ______(b)______:
        return 1
    else:
        total = 0
        i = ____(c)____
        while i < n:
            total += ____(d)____
            i += 1
    return total
```

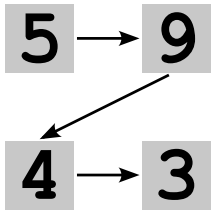
◯ None of the options

◯ Both options

Question 6: New Year's Resolution: Getting in Shape (40 points)

Write a Python function `reshape(arr, rows, cols)` that takes as input a one-dimensional list `arr` and two positive integers `rows, cols`. The function should reshape the input list into a NumPy array with a number of rows equal to `rows` and a number of columns equal to `cols` without changing its data. The function should return the reshaped NumPy array. All the elements from the input list need to fit into the new array. In addition, the order of the elements in the input list should be preserved such that the sequence of elements goes along rows first. Once you reach the end of a row, you'll continue from the beginning of the next row. See the schematic below for an example.

```
>>> arr = [5, 9, 4, 3]
>>> reshape(arr, 2, 2)
[[5, 9],
 [4, 3]]
```



Since all elements in the input list should appear in the new array, not all inputs `rows, cols` are compatible with the input list. If the inputs are incompatible, the function should raise a `ValueError`. There are different ways to implement this function. This method will require you to assign values from the input list into the new array `result` one value at a time.

```
import numpy as np
def reshape(arr, rows, cols):
    """Reshape a list into a NumPy array without changing its data.
    >>> reshape([5, 9, 4, 3], 3, 2)
    ValueError: Input array length does not match input dimensions.

    >>> reshape([5, 9, 4, 3, 1, 7], 3, 2)
    [[5, 9],
     [4, 3],
     [1, 7]]

    >>> reshape([5, 9, 4, 3, 1, 7], 2, 3)
    [[5, 9, 4],
     [3, 1, 7]]

    """
    if len(arr) ______(a)______:
        raise ValueError("Input array length does not match input dimensions.")
    result = ______(b)______ # initialize new array with the correct dimensions
    index = 0                # keep track of the current element in arr
    for i in ______(c)______:
        for j in ______(d)______:
            ______(e)______
            index += ______(f)______
    return result
```

You will implement the function `reshape` by filling in the blanks. You may only write one statement on each line. You may not use ; to place two statements on the same line. You may not use ANY functions or methods, built-in or otherwise, unless given as multiple choices or needed for iteration. You may not modify the level of indentation of any line.

(a) (5 pt) Fill in blank (a).

```
if len(arr)
```

(b) (6 pt) Which of the following options could fill in blank (b)? Select ALL that apply.

- ☐ `np.array((rows, cols))`
- ☐ `np.zeros((rows, cols))`
- ☐ `np.ones((rows, cols))`
- ☐ `np.empty((rows, cols))`
- ☐ `np.array((cols, rows))`
- ☐ `np.zeros((cols, rows))`
- ☐ `np.ones((cols, rows))`
- ☐ `np.empty((cols, rows))`

(c) (5 pt) Fill in blank (c).

```
for i in
```

(d) (5 pt) Fill in blank (d).

```
for j in
```

(e) (6 pt) Fill in blank (e).

```

```

(f) (3 pt) Fill in blank (f).

```
index +=
```

STOP! READ THE TWO LINES BELOW CAREFULLY!

The last part of the exam is only worth 10 out of 300 points (3.3 %), and some might find it challenging. Finish the rest of the exam first!

Next, you will implement the same function `reshape(arr, rows, cols)` but using two different methods. This second method will require you to assign each column (one entire column at a time) instead of assigning one value at a time. Assume that the inputs are compatible, and therefore, *you do not need* to check them or raise a `ValueError`.

```
import numpy as np
def reshape(arr, rows, cols):
    """Reshape a list into a NumPy array without changing its data.
    >>> reshape([5, 9, 4, 3, 1, 7], 3, 2)
    [[5, 9],
     [4, 3],
     [1, 7]]

    >>> reshape([5, 9, 4, 3, 1, 7], 2, 3)
    [[5, 9, 4],
     [3, 1, 7]]

    """
    # initialize new array with the correct dimensions
    result = # (same answer as part b)

    for i in ______(g)______:

        ______(h)______

    return result
```

You will implement the function `reshape` by filling in the blanks. You may only write one statement on each line. You may not use ; to place two statements on the same line. You may not use ANY functions or methods, built-in or otherwise, unless given as multiple choices or needed for iteration. You may not modify the level of indentation of any line.

(g) (1 pt) Fill in blank (g).

```
for i in
```

(h) (4 pt) Fill in blank (h).

```

```

This third method will require you to assign each row (one entire row at a time) instead of assigning one value at a time. Assume that the inputs are compatible, and therefore, *you do not need* to check them or raise a `ValueError`.

```
import numpy as np
def reshape(arr, rows, cols):
    """Reshape a list into a NumPy array without changing its data.
    >>> reshape([5, 9, 4, 3, 1, 7], 3, 2)
    [[5, 9],
     [4, 3],
     [1, 7]]
    >>> reshape([5, 9, 4, 3, 1, 7], 2, 3)
    [[5, 9, 4],
     [3, 1, 7]]
    """
    # initialize new array with the correct dimensions
    result = # (same answer as part b)
    for i in ______(i)______:
        ______(j)______
    return result
```

You will implement the function `reshape` by filling in the blanks. You may only write one statement on each line. You may not use ; to place two statements on the same line. You may not use ANY functions or methods, built-in or otherwise, unless given as multiple choices or needed for iteration. You may not modify the level of indentation of any line.

(i) (1 pt) Fill in blank (i).

```
for i in
```

(j) (4 pt) Fill in blank (j).

```

```